

Python Environment Debugging: What Happened and Why

scKNIFE_graph project — venv, PATH, and VS Code interpreter troubleshooting

Prepared 2026-07-08

Contents

1. Summary
2. Part 1 — Theory: what these pieces actually are
 1. Virtual environments
 2. PATH and command resolution
 3. Shell startup files, in order
 4. Homebrew's naming convention
 5. How an IDE picks a Python interpreter
3. Part 2 — What was actually wrong
4. Part 3 — Every fix, file by file
5. Part 4 — Verifying health / future troubleshooting

Summary

The symptom was simple: `import requests` failed inside an "activated" virtual environment (`venv`), even after repeated `pip install requests` . The actual cause was three independent, stacked problems, each hiding the next:

1. The venv's own `python / python3` executables — which are just symlinks, not real copies of Python — had been silently re-pointed at the wrong system interpreter, so the venv could no longer see its own installed packages.
2. That re-pointing was possible because of a years-old leftover from a python.org 3.10 installer, which had inserted itself into **two independent layers** of macOS's PATH resolution — a shell profile line and a set of root-owned files in `/usr/local/bin` — either of which alone would have been fixable, but together kept resurrecting the problem.
3. Even after PATH was fully corrected, VS Code's Run button still used the wrong interpreter, because the editor had never been told the venv existed — it was configured to auto-detect from "system," which is exactly the layer that was broken.

All three are now fixed. This document explains the theory behind each layer, then walks through exactly what was found and changed.

Part 1 — Theory: what these pieces actually are

1.1 Virtual environments (venv)

A Python virtual environment is *not* a separate copy of Python. It's a small directory tree that borrows a real ("system" or Homebrew-installed) Python interpreter but gives itself an isolated place to install packages, so that installing a package for one project doesn't affect any other project or the system Python. A venv directory looks like this:

```

venv/
├── pyvenv.cfg          <- records which interpreter this venv was built from ("home")
├── bin/
│   ├── activate       <- shell script: modifies PATH and prompt for this shell session
│   ├── python          -> python3              (symlink chain)
│   ├── python3         -> python3.12          (symlink chain)
│   ├── python3.12      -> /opt/homebrew/opt/python@3.12/bin/python3.12 (real interpreter)
│   ├── pip, pip3, pip3.12 <- similarly symlinked / small wrapper scripts
└── lib/
    └── python3.12/
        └── site-packages/ <- where "pip install X" actually puts files, e.g. requests/

```

Two details matter for this story:

- **The interpreter binaries in `venv/bin/` are usually symlinks, not copies.** This saves disk space, but it means the venv has no independent existence — it fundamentally trusts that `venv/bin/python3` keeps pointing at the same real interpreter it was built with. If something re-points `venv/bin/python3` at a *different* Python installation, that different installation has its own, separate `site-packages` — so every package you installed into the venv effectively "disappears," even though the files are still sitting on disk at `venv/lib/python3.12/site-packages`.
- **`pyvenv.cfg` records the interpreter the venv was created from** (its "home"). Tools can use this file to sanity-check whether a venv's symlinks still agree with how it was built — a mismatch between `pyvenv.cfg` and what `venv/bin/python3` actually resolves to is a reliable smoking gun for exactly the corruption described above.

Activation (`source venv/bin/activate`) doesn't do anything magic — it just prepends `venv/bin` to your shell's `PATH` for the current session and sets a `VIRTUAL_ENV` variable. So typing `python` after activating finds `venv/bin/python` first. This only helps if `venv/bin/python` itself is pointed at the right place — activation can't fix a venv whose own symlinks are already broken.

1.2 PATH and command resolution

`PATH` is an ordered, colon-separated list of directories. When you type a bare command name like `python3` (no slash in it), the shell searches those directories **left to right** and runs the first executable file it finds with that exact name. It never searches all of them and picks the "best" one — first match wins, permanently, until `PATH` changes.

This is why the same command name (`python3`) can mean completely different, unrelated installations of Python depending on which directory happens to list it first.

1.3 Shell startup files, in order

Every time a new interactive login shell starts (a new Terminal tab, a new IDE-integrated terminal, etc.), `zsh` reads a fixed sequence of files, each of which can further modify `PATH`:

```

/etc/zshenv -> ~/.zshenv -> /etc/zprofile -> ~/.zprofile
            -> /etc/zshrc -> ~/.zshrc -> /etc/zlogin -> ~/.zlogin

```

Whatever runs *later* in this chain can override whatever an earlier file set up — including re-prepending a directory that an earlier file had already de-prioritized. On top of the dotfiles, macOS itself injects two more layers via a helper program:

- `/etc/paths` — a plain text file, one base directory per line (e.g. `/usr/local/bin`, `/usr/bin`, `/bin` ...). These are *always* on `PATH`, and `/usr/local/bin` is listed first.
- `/etc/paths.d/*` — a directory of small text files (Homebrew installs one named `homebrew` here, containing `/opt/homebrew/bin`), merged in alphabetically *after* the base `/etc/paths` entries.

A program called `path_helper` reads both of these and merges them into `PATH`. On this Mac, `path_helper` is invoked twice — once from `/etc/zprofile` and again from `/etc/zshrc` — meaning system-level directory ordering gets re-applied partway through shell startup, after `~/zprofile` has already run but before `~/zshrc` gets its turn.

1.4 Homebrew's naming convention for Python

Homebrew's `python@3.12` formula deliberately installs *only* version-suffixed binaries — `python3.12`, `pip3.12` — and intentionally **does not** provide unversioned `python3` or `pip3` in `/opt/homebrew/bin`. This is by design (Homebrew's own package notes confirm it: "Python is installed as `/opt/homebrew/bin/python3.12`"). The reasoning is to avoid silently overriding whatever a user or other tool already assumes an unversioned `python3` means.

The practical consequence: no amount of `PATH` reordering can ever make a bare `python3` resolve to Homebrew's build, because Homebrew never enters that particular name into the race at all. Some *other* installation of Python — whichever one does provide an unversioned `python3` — will always win for that exact command name, regardless of directory order.

1.5 How an IDE picks a Python interpreter

VS Code (and this project's fork, Antigravity) delegates interpreter selection to the Python extension(s). The newer "Python Environments" extension stores its configuration per-workspace in `.vscode/settings.json`, primarily through two settings:

- `python-envs.defaultEnvManager` — which strategy the extension uses to discover interpreters (e.g. `system` = whatever's resolvable like a shell would find it via `PATH`, vs. `venv` = look for/prefer a local virtual environment).
- `python-envs.pythonProjects` — an explicit list associating a folder in the workspace with a specific environment manager, overriding the default per-project.

The classic `ms-python.python` extension also honors `python.defaultInterpreterPath`, a direct absolute (or `${workspaceFolder}`-relative) path to the interpreter the Run button and integrated terminal should use, independent of any `PATH`-based discovery. Setting this explicitly removes ambiguity entirely — the editor no longer has to "guess" from a potentially-broken system `PATH`.

Part 2 — What was actually wrong

Finding 1 — The venv's `python3` symlink pointed at the wrong interpreter

`venv/pyvenv.cfg` recorded that the venv was built from Homebrew's Python 3.12 (`/opt/homebrew/opt/python@3.12/bin/python3.12`), and `requests 2.34.2` was correctly installed at `venv/lib/python3.12/site-packages`. But inside `venv/bin/`:

```
venv/bin/python      -> python3
venv/bin/python3     -> /Library/Frameworks/Python.framework/Versions/3.10/bin/python3   (WRONG)
venv/bin/python3.12  -> /opt/homebrew/opt/python@3.12/bin/python3.12                 (correct, unused)
```

Running `python` or `python3` inside this "activated" venv actually launched the system's separate Python 3.10 install, which has its own unrelated `site-packages` and had never heard of `requests`. All four symlinks shared the same timestamp, but their targets didn't make sense together — a normal `python -m venv` run always makes `python3` point at its sibling `python3.12` in the same directory, not at an unrelated absolute path. This is the signature of an external tool re-linking an existing venv after the fact (see Finding 3).

Finding 2 — Two independent PATH problems, both leftover from a python.org installer

Root cause. A python.org Python 3.10 installer (installed Jan 23) had inserted itself into the system in two separate, independent ways. Fixing only one of them would not have been enough.

Layer	What it contained	Why it mattered
~/.zprofile	An explicit line: PATH="/Library/Frameworks/Python.framework/Versions/3.10/bin:\${PATH}" , added directly under the comment "Setting PATH for Python 3.10"	Unconditionally prepended the 3.10 framework's bin directory in front of everything Homebrew had just set up one line above it.
/usr/local/bin/*	A full set of root-owned symlinks — python3 , pip3 , python3.10 , idle3 , 2to3 , etc. — all pointing into the same 3.10 framework install	/usr/local/bin is listed first in /etc/paths , so macOS's own path_helper always placed it ahead of /opt/homebrew/bin independent of anything in a personal dotfile.

Combined with Finding on Homebrew's naming convention above (§1.4): even after removing the ~/.zprofile line and explicitly prepending /opt/homebrew/bin in ~/.zshrc , a bare python3 still resolved to /usr/local/bin/python3 (the 3.10 build) — because Homebrew simply doesn't offer a competing unversioned python3 anywhere. There was no "more correct" ordering that could fix this; the missing binary had to be supplied.

Finding 3 — VS Code never knew the venv existed

.vscode/settings.json originally contained only:

```
{
  "python-envs.defaultEnvManager": "ms-python.python:system",
  "python-envs.pythonProjects": []
}
```

"system" discovery mode means the extension resolves an interpreter the same way a shell would — by walking PATH — which was exactly the broken layer from Finding 2. With no python.defaultInterpreterPath and an empty pythonProjects list, the editor had no project-specific override to fall back on, so the Run button used whatever system interpreter it found — not the project's venv .

This also explains Finding 1's mechanism: a "system"-mode environment manager, when it re-resolves or "repairs" a known venv's interpreter links, does so by asking the system for python3 — landing on the same broken resolution and silently overwriting venv/bin/python3 to match.

Part 3 — Every fix, file by file

3.1 Removed the stale PATH override

```
~/ .zprofile
```

Deleted the leftover python.org installer block:

```
# removed:
# Setting PATH for Python 3.10
# The original version is saved in .zprofile.pysave
PATH="/Library/Frameworks/Python.framework/Versions/3.10/bin:${PATH}"
export PATH
```

This stops the 3.10 framework build from being unconditionally forced ahead of Homebrew on every new login shell. The 3.10 install itself was left alone — it's still reachable by its full path or as `python3.10` if ever needed; only the automatic PATH-shadowing was removed.

3.2 Reasserted Homebrew's priority generally

```
~/ .zshrc
```

```
# Ensure Homebrew (python@3.12, etc.) wins over python.org's /usr/local/bin shims
export PATH="/opt/homebrew/bin:/opt/homebrew/sbin:${PATH}"
```

Good general hygiene for every other Homebrew-installed tool that *does* provide unversioned binaries — but, per §1.4/Finding 2, insufficient by itself for `python3` specifically, since Homebrew never lists one there to prioritize.

3.3 Supplied the missing unversioned interpreter

```
~/bin/python3, ~/bin/pip3
```

```
ln -sf /opt/homebrew/bin/python3.12 ~/bin/python3
ln -sf /opt/homebrew/bin/pip3.12    ~/bin/pip3
```

This is the fix that actually closes the loop. It creates the one thing Homebrew deliberately doesn't: a plain, unversioned `python3 / pip3` that any PATH-based lookup can find — owned by the user, no `sudo` required, and pointed squarely at Homebrew's 3.12 build.

```
~/ .zshrc (addition)
```

```
export PATH="$HOME/bin:${PATH}"
```

Placed at the very front of `PATH`, ahead of even `/opt/homebrew/bin`, so this personal `python3 / pip3` always wins first. Verified in a fully clean environment (`env -i`, no inherited variables at all):

```
$ type -a python3
python3 is /Users/jerryfan/bin/python3
python3 is /usr/local/bin/python3
python3 is /usr/bin/python3

$ python3 --version
Python 3.12.13
```

3.4 Recreated the venv

venv/

With bare `python3` now correctly meaning 3.12, the venv was rebuilt from scratch (`rm -rf venv && python3 -m venv venv`). Its symlinks now agree with each other and with `pyenv.cfg`:

```
venv/bin/python      -> python3.12
venv/bin/python3     -> python3.12
venv/bin/python3.12 -> /opt/homebrew/opt/python@3.12/bin/python3.12
```

`requests` and its dependencies were reinstalled and confirmed present in `venv/lib/python3.12/site-packages`, matching `requirements.txt` (`certifi`, `charset-normalizer`, `idna`, `requests`, `urllib3`).

3.5 Told the editor where the venv is

`.vscode/settings.json`

Current contents:

```
{
  "python-envs.defaultEnvManager": "ms-python.python:venv",
  "python-envs.pythonProjects": [
    {
      "path": "${workspaceFolder}",
      "envManager": "ms-python.python:venv"
    }
  ],
  "python.defaultInterpreterPath": "${workspaceFolder}/venv/bin/python"
}
```

This pins the workspace to `venv`-based discovery instead of "system," registers this project folder explicitly with the venv environment manager, and sets a direct, unambiguous `python.defaultInterpreterPath` that the classic extension's Run button honors regardless of any PATH state. Combined with selecting the interpreter once via **Cmd+Shift+P** → **Python: Select Interpreter** → `./venv/bin/python`, the Run button and integrated terminal no longer depend on PATH resolution at all for this project.

Part 4 — Verifying health / future troubleshooting

Quick checks if something like this happens again

Command	What it tells you
<code>type -a python3</code>	Every location on PATH providing <code>python3</code> , in the order the shell will pick them. First line is what actually runs.
<code>python3 -c "import sys; print(sys.executable)"</code>	Inside an activated venv, this should print a path <i>inside</i> that venv's <code>bin/</code> . If it doesn't, activation isn't working or the venv's symlinks are broken.
<code>cat venv/pyvenv.cfg</code> vs. <code>ls -la venv/bin/python*</code>	The interpreter recorded in <code>pyvenv.cfg</code> ("home") should match what the <code>venv/bin</code> symlinks actually resolve to. A mismatch means the venv has been silently re-linked.
<code>pip show <package></code> (run via the venv's own <code>pip</code> , e.g. <code>./venv/bin/pip show requests</code>)	Confirms the package is actually installed in that venv's <code>site-packages</code> , independent of whatever <code>python3</code> currently resolves to.

General principle worth keeping

Ambiguous, unversioned command names (`python3` , `pip3`) are inherently fragile in any setup with more than one Python installation, because their meaning depends entirely on PATH order at the moment they're invoked — which can be changed by a shell profile edit, an installer, or (as here) an IDE extension "repairing" something on your behalf. Where possible, prefer being explicit (`python3.12` , or a venv's activated `python`) over relying on the bare name, and treat any tool that silently rewrites interpreter symlinks as something to double-check after major environment changes.